NODE.JS INTERVIEW
REVISION SHEET

120+ Questions Every Backend Developer Should Know

Includes:

- Node.js Fundamentals
- Event Loop
- Async Programming
- Express & Middleware
- Streams & Buffers
- Authentication (JWT)
- Scaling & Clustering
- *System Design Concepts*
- *Scaling & Clustering*
- *System Design Concepts*

Perfect for:

- Backend Developers
- Quick Revision
- Node.js Interviews

Save this post before your next interview.

Q 1. What is Node.js?

Ans: Node.js is a JavaScript runtime built on Chrome's V8 engine that allows running JavaScript outside the browser, mainly on servers.

It is event-driven and non-blocking, which makes it good for scalable applications.

Example: `console.log("Hello from Node.js");`

Run with: `$ node app.js`

Q 2. Why is Node.js fast?

Ans: Node.js uses:

- V8 engine for fast execution
- Non-blocking I/O
- Event-driven architecture

Instead of waiting for tasks like database queies, Node continues processing other requests.

Example concept:

```
Request 1 → DB query  → Request 2
Request 2 → handled   → Handled immediately
```

Q 3. What is V8 Engine?

Ans: **Yes**: V8 is Google's JavaScript enine used in Chrome and Node.js.

It converts JavaScript into machine code, making execution faster.

Example:

```
let sum = 10 + 20;
console.log(sum);
```

Q 4. Is Node.js single-threaded?

Ans: **Yes**: Node.js uses a single-threaded event loop.

But heavy operations can be handled by worker threads internally.

Q 5. Where is Node.js used?

Ans: Example:

- REST APIs
- Streaming services
- Microservices
- Real-time apps (chat)
- Microservices
- Server-side rendering.

Q 6. What is npm?

Ans: npm is Node Package Manager used to install libraries.

Example install:

```
npm install express
```

```
npm install -g nodemon
```

Q 7. What is package.json?

Ans: It stores:

- project information
- dependencies
- scripts
- version info

Example:

```
{"name": "node-app",
 "version": "1.0.0",
 "dependencies"{
  "express": "^4.18.0"
 }
```

Q 8. What is npx?

Ans: npx allows running packages without installing globally.

Example:

```
npx create-react-app myapp
```

Q 9. What are Node.js modules?

Ans: Modules are reusable pieces of code.

Types:

- Core modules
- Third-party modules

- **Core modules**
- **Local modules**
- **Third-party modules**

Q 10. What are core modules?

Ans: Built-in modules provided by Nde.js.

Common ones:

- http
- path
- crypto
- fs
- os
- events

```
const os = require("os");
console.log(os.platform());
```

11. What is the fs module?

Ans: fs stands for File System.

Used for reading, writing and deleting files.

Example:

```
const fs = require("fs");
```

```
fs.readFile("data.txt","utf8",(err,data) => {
  console.log(data);
};
```

12. What is require()?

Ans: require() is used to import modules.

Example:

```
const htp = require("http");
```

13. What is module.exports?

Ans: Used to export functions or variables from a module.

Example:

math.js

```
function add(a,b){
  return a+b;
}
module.exports = add;
```

app.js

```
const add = require("/.math");
console.log(add(2,3));
```

14. What is ES Module import?

Ans: Modern way of importing modules.

Example:

```
import fs from "fs";
```

Used when: "type": "module" in package.json.

15. Difference between require and import?

	require	import
CommonJS	CommonJS	ES Modules
Dynamic	Dynamic	Static
Used in Node older versions		ModernJS

16. What is Event Loop?

Ans: The Event Loop is the core mechanism that allows Node.js to handle multiple requests using a single thread.

Instead of creating a new thread for every request, Node.js places incoming operations in a queue and process them asynchronously.

The event loop continuously checks whether tasks are completed and then executes their callbacks.

Process flow; Request → Event Loop → Callback Queue → Response

This architecture makes Node.js highly scalable for handling many concurrent connections.

17. What is Non-blocking I/O?

Ans: Non-blocking I/O means Node.js does not wait for a task to finish before moving to the next operation. When an operation such as reading a file or querying a database is started, Node.js continues executing other tasks while waiting for the result.

Once the operation is completed, the result is returned through a callback or promise.

This approach improves performance because the server can handle many requests simultaneously.

18. What is Blocking Code?

Ans: Blocking code stops the execution of the program until the current task finishes. During this time, Node.js cannot process any other operations.

Blocking operations are generally slower in server environments because they prevent the system from handling multiple requests efficiently.

For high-performance applications, non-blocking operations are usually prefered.

20. What is Callback Hell?

Ans: Callback Hell occurs when multiple callbacks are nested inside one another. This makes the code difficult to read, maintain, and debug.

When many asynchronous operations depend on each other, deeply nested callbacks create messy and confusing code structures.

Common solutions to callback hell include:

- Using Promises
- Using Async/Await
- Breaking code into smaller reusable functions

21. What is a Promise?

Ans: Promise represents a future value.

States:

- Pending
- Resolved
- Rejected

Example:
```
const p = new Promise((resolve,reject) → {
resolve("Success");
});
```

Example:
```
fetchData().then(data → console.log(data))
.catch(err → console.log(err));
```

22. Promise methods

Ans:
```
.then()
.catch()
.finally()
```

Example:
```
fetchData()
.then(data → console.log(data))
.catch(err → console.log(err));
```

23. What is async function?

Ans: Declares asynchronous function.

Example:
```
async function getData()
return "Hello";
```

24. What is await?

Ans: Used to wait for promise result.

Example:
```
const data = await fetchData();
```

25. Advantages of async/await

Ans: Callback is a function that is passed as an argument to another function and is executed after a task completed.

Callbacks are commonly used in Node.js to handle asynchronous operations such as file reading, database queries, and APi requests.

They allow Node.js to continue executing other

- Using Promises
- Using Async/Await
- Breaking code into smaller reusable functions

26 What is the HTTP module in Node.js?

Ans: The **http** module is used to create web servers and handle HTTP requests.

Example:

```
const http = require("http");
const server = http.createServer({req, res) => {
    res.write("Hello Server");
    res.end();
});
server.listen(3000);
```

27 What is Express.js?

Ans: Express is a minimal web framework for Node.js that simplifies building APIs and web applications.

Example:

```
const express = require("express");
const app = express();
    app.get("/  (req, res ) => {
        res.send("Hello Express");
    });
app.listen(3000);
```

28 What is Routing in Express?

Routing means defining how the server responds to different endpoints.

Example:

```
app.get("/users", (req,res)=>{
    res.send("Users list")
});
```

30 What are HTTP Status Codes?

Used to indicate request results.
Common ones:

Code	Meaning
200	Success
201	Created
400	Bad Request
404	Not Found
500	Server Error

29 What are HTTP Methods?

Common REST methods:

Method	Purpose
GET	Fetch data
POST	Create data
PUT	Update full resource
PATCH	Update partial
DELETE	Remove data

Example:

```
app.post("/user".(req,res)=>{
    res.send("User created");
```

Example:

```
res.status(200).json(
 {message:"success" ");
```

31. What is Middleware in Express?

Ans: Middleware functions execute before the final request handler.

Example:

```
app.use((req, res, next) => {
        console.log("Request received");
        next();
});
```

32. Types of Middleware

Ans: Application middleware

- Router middleware
- Error middleware
- Third-party middleware

Example: helmet, cors, morgan, body-parser

- helmet
- cors
- cors
- morgan
- body-parser

33. What is body-parser?

Ans: Used to parse request body data.

Example: app.use(express.json();

Now request body is accessible:

- req.body

34. What is CORS?

Ans: CORS allows cross-origin requests between different domains.

Example:

```
const cors = require("cors")
app.use(cors))
```

35. What is Helmet?

Ans: Helmet secures Node.js apps by setting HTTP headers.

Example:

```
const helmet = require("helmet");
app.use(helmet()
```

36. What is a Stream in Node.js?

Ans: Middleware functions execute before the final request handler.

Example:

```
const fs = require("fs");
const stream = fs.createReadStream( "file.txt")
stream.on("data", (chunk) => {
        console.log(chunk);
}});
```

37. Types of Streams

Ans: Readable

- Writable
- Duplex
- Transform

Example:

- fs.createReadStream()
- fs.createWriteStream()

38. What is Pipe in Streams?

Ans: Pipe transfers data from one stream to another.

Example:

```
const read = fs.createReadStream("input.txt");
const write = fs.createWriteStream("output.txt");
read.pipe(write);
```

39. What is Buffer in Node.js?

Ans: Buffer stores binary data temporarirly.

Example:

```
const buffer = Buffer.from("Hello");
console.log(buffer);
```

40. Why Buffers are needed?

Node.js handles binary streams, like fills, files, videos, and network data, so buffers store data before processing.

Q 41. What is EventEmitter?

Ans: It allows creating custom events in Node.js.

Example:

```
const EventEmitter = require("events");
const emitter = new EventEmitter();
emitter.on("greet", () => {
        console.log("Hello event triggered");
}};
emitter.emit("greet")
```

Q 42. What is process object?

Ans: process gives information about the current Node process.

Example:

```
console.log(process.version)
```

Q 43. What is process.env?

Ans: Used for environment variables.

Example:

```
console.log(process.env.PORT);
```

Often used with dotenv.

Q 44. What is dotenv?

Ans: dotenv loads environment variables from a .env file.

Example .env file:

```
PORT=5000
DB_URL=mongodb://localhost
```

Usage:

```
require("dotenv").config()
```

Q 45. What is nodemon?

Ans: Nodemon automatically restarts server when files change.

Install:

```
npm install -g nodemon
```

Run:

```
nodemon app.js
```

Q 46. What is JWT?

Ans: JWT (JSON Web Token) is used for authentication.

Structure:

Header.Payload.Signature

Q 47. How to create JWT?

Ans: Example:

```
const jwt = require("josohtoken`n');
```

```
const token = jwt.sign((id:1), 'secret');
```

Q 48. How to verify JWT?

Ans:

```
jwt.verify(token, "secret');
```

Q 49. What is Authentication vs Authorization?

Ans: Authentication → Who you are

Authorization → What you can access

Example:

- Login → authentication
- Access admin panel → authorization

Q 50. What is Rate Limiting?

Ans: Prevents too many requests from one user.

Example:

```
const rateLimit = require("express-rate-limit');
```

Used for DDoS protection.

Q 51. What is Clustering in Node.js?

Ans: Node.js uses one CPU core by default.

Cluster allows using multiple CPU cores.

Example:

```
const cluster = require("cluster");
```

Q 52. What are Worker Threads?

Ans: Worker threads run CPU-intensive tasks in background threads.

Example:

- image processing
- data compression

Q 53. What is Load Balancer?

Ans: Distributes traffic across multiple servers.

Example:

- NGINX
- AWS ELB

Q 54. What is PM2?

Ans: PM2 is a Node.js process manager.

Example:

- pm2 start app.js
- Features:
  - clustering
  - monitoring
  - auto restart

Q 55. What is Caching?

Ans: Caching stores frequently used data.

Example:

```
const rateLimit = require("express-rate-limit');
```

Used for DDoS protection.

Q 56. What is Microservices Architecture?

Ans: Application is divided into small independent services.

Example:

- User Service
- Payment Service
- Order Service

Q 57. What is Message Queue?

Ans: Message queues enable asynchronous communication between services.

Examples:

- RabbitMQ
- Kafka
- SQS

Q 58. What is WebSocket?

Ans: WebSocket provides real-time communication.

Examples:

- chat applications
- live notifications

Q 59. What is GraphQL?

Ans: GraphQL is an alternative to REST APIs where client requests exact data needed.

Example:

```
query {
    user {
        name
}
```

Libraries:

- socket.io
- ws

Q 60. What is Reverse Proxy?

Ans: A reverse proxy forwards client requests to backend servers.

Example:

- NGINX
- Cloudflare

Q 61. What is libuv in Node.js?

Ans: libuv is the C library used by Node.js to handle:

- Event loop
- Asynchronous I/O
- Thread pool
- File system operations

It allows Node.js to perform non-blocking operations.

Example concept:

```
Node.js → libuv → OS APIs
```

Q 62. What is the Thread Pool in Node.js?

Ans: Node.js uses a thread pool (default size: 4) to run heavy operations like:

- File system tasks
- DNS lookup
- Crypto operations

Example:

```
const crypto = require("crypto");
crypto.pbkdf2("pass","salt",1000,64,""sha512"),(...){
    console.log("done");
},
```

Q 63. What is Blocking vs Non-Blocking Code?

Ans: Blocking code stops execution until task completes.

Example (blocking):

```
const data = fs.readFileSync("file.txt");
```

Non-blocking code executes asynchronously.

```
fs.readFile("file.txt",(err, data)-> {
    console.log(data);
},
```

Q 64. What is process.nextTick()?

Ans: process.nextTick() executes a callback before the next event loop cycle.

Q 65. What is setImmediate()?

Ans: Executes code after I/O events in the event loop.

Q 66. What is the REPL in Node.js?

Ans: REPL stands for:

- Read
- Eval
- Print
- Loop

It allows executing JavaScript directly in the terminal.

Example:

```
node
> 2 + 3
5
```

Q 67. What is Global Object in Node.js?

Ans: The global object provides variables accessible everywhere.

Example:

```
console.log(__dirname);
console.log(__filename);
```

Output example:

```
/Users/project/app
```

Q 68. What is __dirname?

Ans: __dirname returns the current directory path.

Example:

```
console.log(__dirname);
```

Output example:

```
/Users/project/app
```

Q 69. What is __filename?

Ans: Returns the current file path.

Example:

```
console.log(__filename);
```

Q 70. What is Path Module?

Ans: The path module helps work with file paths.

Example:

```
const path = require("path");
console.log(path.basename(__filename);
```

Output: app.js

Q 71. What is Compression Middleware?

Ans: Used to reduce response size.

Example:

```
const compression = require("compression");
app.use(compression();
```

Benefits:

- faster responses
- reduced bandwidth

Q 72. What is Morgan?

Ans: Morgan is a logging middleware.

Example:

```
const morgan = require("morgan");
app.use(morgan("dev");
```

Logs every HTTP request.

Q 73. What is Logging in Node.js?

Ans: Logging helps track system behavior.

Popular tools:

- Winston
- Morgan
- Pino

```
console.log("Server started");
```

Q 74. What is Error Handling Middleware?

Ans: Used to handle errors globally.

Example:

```
app.use((err,req,res,next)>{
    res.status(500).send("Server Error");
};
```

Q 75. What is Graceful Shutdown?

Ans: Graceful shutdown ensures server finishes current requests before stopping.

Q 76. What is Redis?

Ans: Redis is an in-memory data store used for caching.

Example usage: API request → check Redis → DB

- faster responses
- reduced database load

Q 77. What is CDN?

CDN (Content Delivery Network) distributes content across global servers.

Example: Cloudflare AWS CloudFront

- Used for:
  - static files
  - images
  - videos

Q 78. What is Load Balancing?

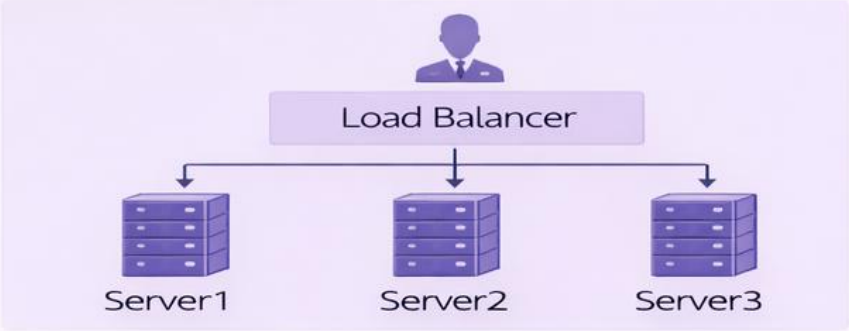
Load balancers distribute traffic across servers.

Example architecture:

- Client
- Load Balancer
- Server1

Example tools:

- Kong
- AWS API Gateway



Q 79. What is API Gateway?

API gateway manages multiple backend services.

- Responsibilities:
- authentication
- rate limiting
- routing

Example tools:

- AWS API Gateway

Example load balancer.

Q 80. What is Horizontal Scaling?

Adding more servers to handle load.

Q 81. What is Vertical Scaling?

Ans: Increasing CPU or RAM of one server.

Example:

4GB RAM → 16GB RAM

Q 82. What is Memory Leak?

Memory leak occurs when **memory is not released** after use.

Common causes:

- global variables
- unused timers
- large caches

Q 83. How to Detect Memory Leaks?

Tools:

Node Inspector | Chrome DevTools | clinic.js

Q 84. What is Heap Memory?

Heap stores dynamic data like:

- objects
- arrays

Example:

```
let user = {name:"John"};
```

Q 85. What is Garbage Collection?

Garbage collector automatically removes unused memory.

Node uses V8 GC.

Server1 ⟷ Server2 ⟷ Server3

Q 86. What is Serverless Architecture?

Ans: **Serverless** means running code without managing servers.

Example:

- AWS Lambda
- Firebase Functions

Q 87. What is Cold Start?

Cold start happens when a serverless function starts for the first time. It may cause small delay.

Q 88. What is Containerization?

Containers package app with **dependencies**.

Example:

- Docker

Q 89. What is Docker?

Docker creates isolated environments.

Example command:

```
$ docker build _
```

Q 90. What is CI/CD?

CI/CD automates build and deployment.

Examples:

- GitHub Actions
- Jenkins
- GitLab CI

Q 91. What is Monitoring?

Ans: Monitoring tracks application performance.

Tools:

Prometheus | Grafana | Datadog

Q 92. What is Health Check Endpoint?

Used to check server status.

```
app.get("/health",(req,res)~{
;res.send("Server OK"); }};
};
```

Q 93. What is API Versioning?

Maintaining multiple API versions.

Example:

```
/api/v1/users
/api/v2/users
```

Q 94. What is Pagination?

Pagination limits results returned from API.

Example:

```
/users?page=1&limit=10
```

Q 95. What is Throttling?

Throttling limits request rate.

Example:

```
100 requests per minute
```

Q 96. What is Webhook?

Ans: **Webhook** sends real-time HTTP callback when an event occurs.

Example:

```
Payment success ⟶ webhook ⟶ upodate order
```

(You already implemented webhooks in your **Next.js** project).

Q 97. What is Idempotency?

Same request produces same result every time.

Example:

```
PUT request
```

Q 98. What is API Documentation?

Documentation explains API usage.

Tools:

Swagger | Postman | Redoc

Q 99. What is Rate Limiting?

Restricts number of requests.

Example:

```
express-rate-limit
```

Q 100. What makes Node.js scalable?

Node.js scalability comes from:

- Event loop
- Non-blocking I/O
- Microservices architecture
- Horizontal scaling

Q 100. What makes Node.js scalable?

Node.js scalability comes from:

- Event loop
- Non-blocking I/O
- Microservices architecture

Q 101. What are the Event Loop Phases in Node.js?

The **Event Loop** processes asynchronous operations in phases.

Main phases:

- Timers
- Pending callbacks
- Idle / Prepare
- Poll
- Check

Timers ⟶ webhook ⟶ update order

Poll ⟶ Check

Close callbacks

Example: `setTimeout(_) ⟶ console.log("Timer"), 0);`

The event loop decides which executes first.

Q 102. Worker Threads vs Cluster

Both are used for **scaling** Node.js applications.

Feature	Worker Threads	Cluster
Purpose	CPU tasks	Multiple processes
Memory	Shared	Separate
Use case	heavy computation	high traffic servers

Example cluster usage:

```
const cluster = require("cluster");
if (cluster.isMaster){
  cluster.fork();
}}
```

Q 103. Streams vs Buffers

Both are used for scaling Node.js applications.

Feature	Worker Threads	Cluster
Purpose	CPU tasks	Multiple processes
Use case	heavy computation	high traffic servers

Example cluster usage:

```
const cluster = require("cluster");
```

Q 104. Express vs Fastify

Both are Node.js frameworks.

Express	Fastify
Older	Newer
Large ecosystem	Faster performance
More middleware	Built-in validation

Q 105. REST vs GraphQL

REST example:

REST	GraphQL
Multiple	Single endpoint
Over-fetching	Fetch exact data
Uses HTTP methods	Uses queries

Q 106. Redis vs Memory Cache

Ans:

Redis	Memory Cache
External store	Inside server
Persistent	Temporary
Scalable	Limited
Example: Redis use: API request → check Redis → DB	

Q 107. JWT vs Session Authentication

Ans:

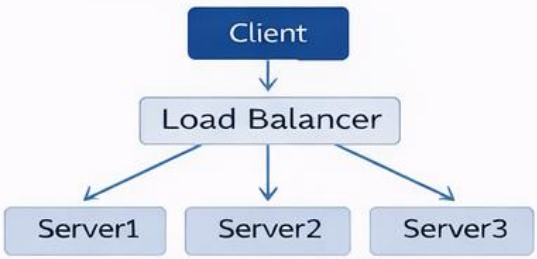
JWT	Session
Stateless	Stateful
Stored on client	Stored on server
Good for APIs	Good for traditional apps
JWT example: `const token = jwt.sign(( id: 1 ), "secret');`	

Q 108. Load Balancing Strategies

Ans: Common strategies:

- Round Robin
- Least Connections
- IP Hash

Example architecture:



Tools: NGINX • AWS ELB

Q 109. Message Queues in Node.js

Ans: Message queues allow asynchronous communicate between services. Used in microservices.

Examples:

- RabbitMQ
- Kafka
- AWS SQS

Example WebSocket use:

- Chat apps
- Live notifications
- Gaming

Q 110. WebSockets vs HTTP

Ans:

HTTP	WebSockets
Request-response	Real-time connection
Stateless	Persistent

Example WebSocket use:

- Chat apps
- Live notifications
- Gaming

Q 110. WebSockets vs HTTP

HTTP	WebSockets
Request-response	Real-time connection
Stateless	Persistent

Example WebSocket use:

- Chat apps
- Live notifications
- Gaming

Q111 Serverless vs Containers

Serverless	Containers
No server management	Runs inside container
Auto scaling	Manual scaling

- CDN distributes content across global servers.

Q112 What are CDN Benefits?

CDN distributes content across global servers.

Benefits:

- faster load time
- reduced server load
- improved performance

Q113 Monitoring Tools for Node.js

Monitoring helps track system performance,

Popular tools:

- Prometheus
- Grafana
- Datadog
- New Relic

Metrics tracked:

- CPU usage
- Memory
- API response time
- API response time

Q114 How to Debug Node.js Applications?

Techniques tools:

- Node Inspector
- Chrome DevTools
- VS Code Debugger

Q115 Node.js Performance Optimization

Techniques:

- caching with Redis
- clustering
- load balancing
- database indexing

Example **compression middleware:**

```
const compression = require("compression");
app.use(compression();
```

116 Node.js Security Best Practices

Important practices:

- Use Helmet
- Validate inputs
- Use HTTPS
- Prevent SQL injection
- Rate limiting

Example:

```
const helmet = require("helmet");
app.use(helmet();
```

117 Scaling Microservices

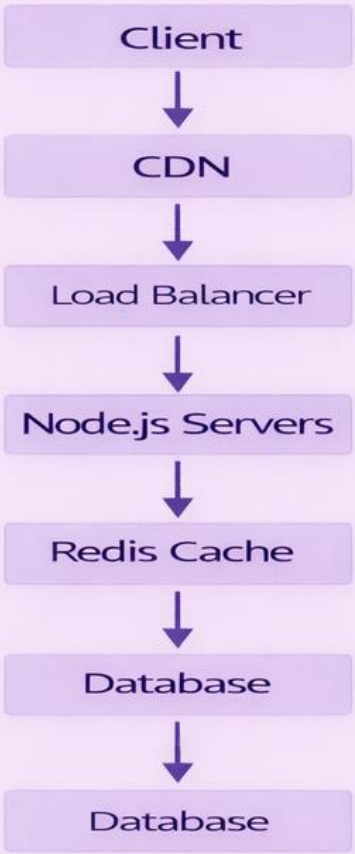
Microservices scale independently.

Example architecture:

- User Service
- Order Service
- Payment Service
- Notification Service

Benefits:

- independent deployment
- fault isolation
- scalability



118 API Rate Limiting

Limits number of requests per user.

Example:

- 100 requests per minute

119 Graceful Shutdown in Node.js

Graceful shutdown ensures ongoing requests complete before server stops.

Example:

```
process.on("SIGINT", () => {
  console.log("Server shutting down");
  process.exit();
```

120 What does Production Node.js Architecture look like?

Typical production architecture:

- Client
- CDN
- Load Balancer
- scalability
- reliability
- high performance